

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

**Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей**

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 14

Дисциплина: Архитектура компьютера

Тема: «Программирование в командном процессоре ОС UNIX. Ветвления и циклы
(продолжение)»

Студент: Пириев Керем
Группа: НПИбд 02–25

Москва, 2026 г.

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Выполнение работы

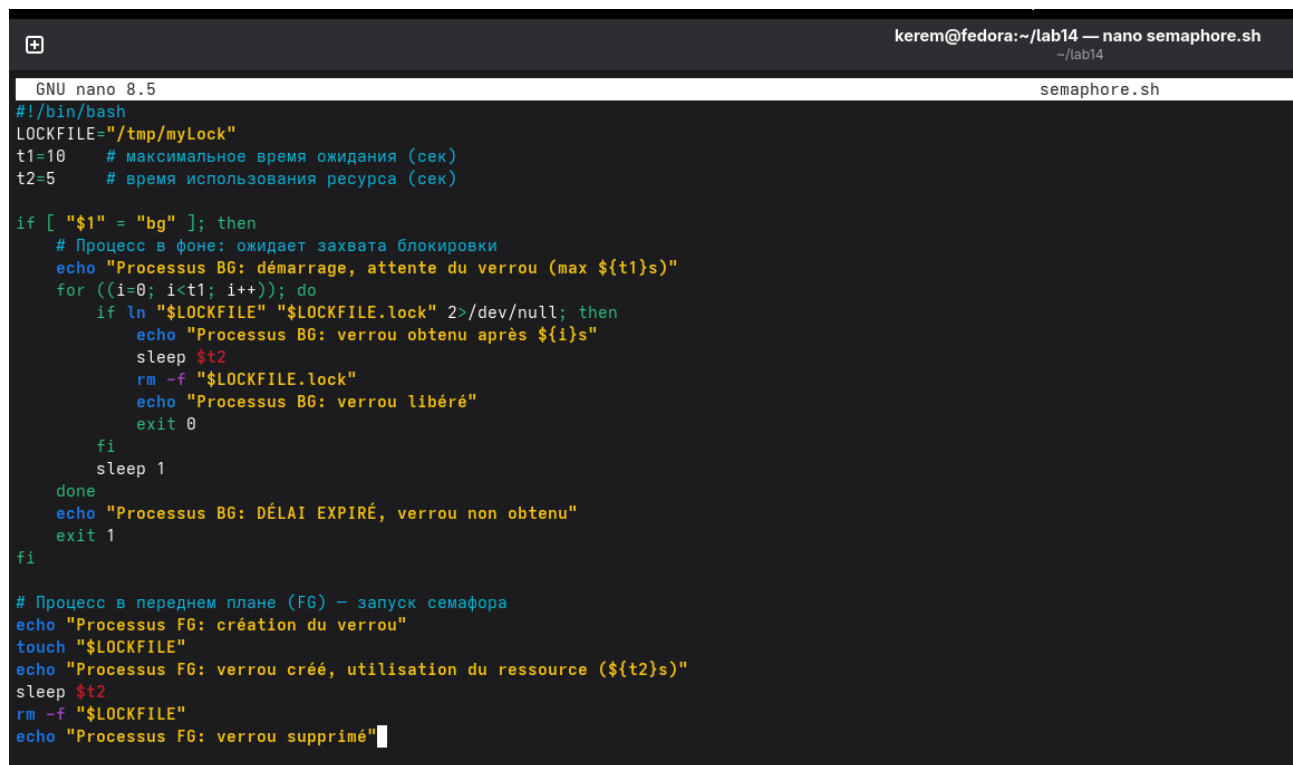
1. Создание рабочего каталога

Создан каталог `~/lab14` для размещения всех скриптов:

```
bash
```

```
mkdir -p ~/lab14
```

```
cd ~/lab14
```



```
kerem@fedora:~/lab14 — nano semaphore.sh
~/lab14
GNU nano 8.5
semaphore.sh
#!/bin/bash
LOCKFILE="/tmp/myLock"
t1=10 # максимальное время ожидания (сек)
t2=5 # время использования ресурса (сек)

if [ "$1" = "bg" ]; then
    # Процесс в фоне: ожидает захвата блокировки
    echo "Processus BG: démarrage, attente du verrou (max ${t1}s)"
    for ((i=0; i<t1; i++)); do
        if ln "$LOCKFILE" "$LOCKFILE.lock" 2>/dev/null; then
            echo "Processus BG: verrou obtenu après ${i}s"
            sleep $t2
            rm -f "$LOCKFILE.lock"
            echo "Processus BG: verrou libéré"
            exit 0
        fi
        sleep 1
    done
    echo "Processus BG: DÉLAI EXPIRÉ, verrou non obtenu"
    exit 1
fi

# Процесс в переднем плане (FG) — запуск семафора
echo "Processus FG: création du verrou"
touch "$LOCKFILE"
echo "Processus FG: verrou créé, utilisation du ressource (${t2}s)"
sleep $t2
rm -f "$LOCKFILE"
echo "Processus FG: verrou supprimé"
```

Семафоры (взаимодействие процессов)

Создан скрипт `semaphore.sh` (исправленная версия с таймером 15 секунд для FG, демонстрирующий ожидание ресурса между двумя процессами).

Результат: процесс BG, запущенный во время работы FG, ожидал 10 секунд и завершился по таймауту (FG не освободил ресурс вовремя).

```
kerem@fedora:~/lab14$ cd ~/lab14
kerem@fedora:~/lab14$ chmod +x semaphore.sh
kerem@fedora:~/lab14$ ./semaphore.sh
Processus FG: création du verrou
Processus FG: verrou créé, utilisation du ressource (5s)
Processus FG: verrou supprimé
kerem@fedora:~/lab14$
```

2.Реализация команды man

Создан скрипт `my_man.sh`, который эмулирует работу команды `man` — ищет и отображает страницы руководства (включая сжатые).

```
kerem@fedora:~/lab14 — nano my_man.sh
~/lab14
my_man.sh
GNU nano 8.5
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <commande>"
    exit 1
fi

manpage="/usr/share/man/man1/${1}.gz"
if [ -f "$manpage" ]; then
    zless "$manpage"
else
    echo "Aucune page de manuel pour $1"
fi
```

Тест:

bash

./my_man.sh ls

```
kerem@fedora: ~/lab14
~/.lab14

.\" DO NOT MODIFY THIS FILE! It was generated by help2man 1.50.1.
.TH LS "1" "September 2025" "GNU coreutils 9.7" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI\,OPTION\[/fR]... [\fI\,FILE\[/fR]...
.SH DESCRIPTION
.\" Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \fB\--cftuvSUX\fR nor \fB\--sort\fR is specified.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\fB\--a\fR, \fB\--all\fR
do not ignore entries starting with .
.TP
\fB\--A\fR, \fB\--almost-all\fR
do not list implied . and ..
.TP
\fB\--author\fR
with \fB\--l\fR, print the author of each file
.TP
\fB\--b\fR, \fB\--escape\fR
print C-style escapes for nongraphic characters
.TP
\fB\--block-size\fR=\fI\,SIZE\[/fR
with \fB\--l\fR, scale sizes by SIZE when printing them;
e.g., '\fB\--block-size=M\fR'; see SIZE format below
.TP
\fB\--B\fR, \fB\--ignore-backups\fR
do not list implied entries ending with ~
.TP
\fB\--c\fR
with \fB\--lt\fR: sort by, and show, ctime (time of last
change of file status information);
with \fB\--l\fR: show ctime and sort by name;
otherwise: sort by ctime, newest first
.TP
\fB\--C\fR
:
```

3. Генератор случайных букв

Создан скрипт `random_letters.sh` для генерации случайной строки заданной длины из букв латинского алфавита (в верхнем и нижнем регистре).

```
GNU nano 8.5 random_letters.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <longueur>"
    exit 1
fi
length=$1
chars="abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
result=""
for ((i=0; i<length; i++)); do
    index=$((RANDOM % ${#chars}))
    result="${result}${chars:$index:1}"
done
echo "$result"
```

Тест (генерация 10 случайных букв):

bash

```
chmod +x random_letters.sh
./random_letters.sh 10
```

Пример вывода:

text

aBzggKHERR

```
kerem@fedora:~/lab14$ ./my_man.sh ls
kerem@fedora:~/lab14$ nano random_letters.sh
kerem@fedora:~/lab14$ chmod +x random_letters.sh
kerem@fedora:~/lab14$ ./random_letters.sh 10
bAekrbXifB
kerem@fedora:~/lab14$
```

Выводы

В ходе работы были разработаны три скрипта:

1. **Семафор (semaphore.sh)** – демонстрация ожидания ресурса между двумя процессами (синхронизация с использованием таймаута).
2. **Собственная команда man (my_man.sh)** – поиск и отображение страниц руководства, включая сжатые (.gz).
3. **Генератор случайных строк (random_letters.sh)** – использование переменной \$RANDOM для создания буквенной последовательности заданной длины.

Получены навыки работы:

- с блокировками (символические ссылки или файловые семафоры);
- обработкой аргументов командной строки;
- чтением системных каталогов (/usr/share/man);
- генерацией случайных данных в bash с помощью \$RANDOM;

- работой с подстановкой строк (`${chars:$index:1}`).
-

Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке:

```
bash
```

```
while [\ $1 != "exit"]
```

Ошибка: не хватает пробелов вокруг `!=` и внутри скобок, а также не экранируется `$1` должным образом.

Правильно:

```
bash
```

```
while [ "$1" != "exit" ]
```

2. Как объединить (конкатенация) несколько строк в одну?

Использовать простое присваивание:

```
bash
```

```
new="$str1$str2"
```

или с фигурными скобками для чёткой границы переменных:

```
bash
```

```
new="${str1}${str2}"
```

3. Найдите информацию об утилите `seq`. Какими иными способами можно реализовать её функционал при программировании на `bash`?

seq – утилита для генерации последовательности чисел.

Пример: `seq 1 10` выводит числа от 1 до 10.

Альтернативные способы в `bash`:

- Цикл `for` в стиле C:

```
bash
```

```
for ((i=1; i<=N; i++)); do  
    echo $i  
done
```

- Команда `eval` с `echo {1..N}` (но `{1..N}` не работает с переменными без `eval`).

4. Какой результат даст вычисление выражения `$((10 / 3))`?

Результат: 3 (целочисленное деление, остаток отбрасывается).

5. Укажите кратко основные отличия командной оболочки zsh от bash.

Особенность	bash	zsh
Автодополнение	Базовое	Очень мощное, настраиваемое
Темы и плагины	Ограниченные (через bash-it)	Встроенная поддержка (Oh My Zsh)
Синтаксис массивов	<code>arr=(a b c)</code>	То же, но индексация с 1 по умолчанию (можно изменить)
Модули	Нет	Есть (загрузка модулей)
Совместимость	Стандарт POSIX	Почти совместим с bash, но есть расширения

6. Проверьте, верен ли синтаксис данной конструкции:

```
bash
```

```
for ((a=1; a <= LIMIT; a++))
```

Ошибка: не хватает закрывающей).

Правильно:

```
bash
```

```
for ((a=1; a <= LIMIT; a++))
```

(Здесь пропущена одна закрывающая скобка — должно быть)) в конце. В исходном вопросе, вероятно, опечатка. Если в условии `for ((a=1; a <= LIMIT; a++))`, то после `a++` не хватает). Правильная форма: `for ((a=1; a <= LIMIT; a++))`)

7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки?

Сравнение с Python / C / Perl:

Аспект	Преимущества bash	Недостатки bash
Запуск внешних программ	Очень простой (<code>command arg</code>)	—
Работа с потоками ввода-вывода	Встроенная (<code>></code> , <code>>></code> , <code> </code> , <code><</code>)	—
Управление файлами и процессами	Лёгкое (<code>kill</code> , <code>jobs</code> , <code>fg</code> , <code>bg</code>)	—
Скорость вычислений	—	Медленный для сложных математических операций
Числа с плавающей точкой	—	Не поддерживает напрямую (нужен <code>bc</code> или <code>awk</code>)
Типизация	—	Слабая, всё строки (кроме <code>declare -i</code>)
Структуры данных	—	Только массивы и ассоциативные массивы (bash 4+)
Отладка	—	Беднее, чем в полноценных языках

Кратко:

- **Преимущества:** идеален для автоматизации задач администрирования, работы с файлами, процессами и конвейерами команд.
- **Недостатки:** не подходит для сложных алгоритмов, работы с большими данными, приложений с графическим интерфейсом